Frill: ML on Z80

A Functional Language for Vintage Hardware

MinZ Project

2026-03-23

Frill: ML on Z80

What Is Frill?

Frill is an ML-style functional language that compiles to Z80 machine code. The name is deliberately ironic: "frills" — decorations, embellishments — on the most constrained hardware imaginable. 64KB RAM. 3.5MHz. No OS.

And yet: algebraic data types, pattern matching, parametric polymorphism, an effect system, property-based testing, pipe operators, and lambda expressions. All compiling to the same tight Z80 assembly a hand-written program would produce.

"What if OCaml ran on a ZX Spectrum?"

Why ML on Z80?

This sounds absurd. ML-family languages (OCaml, Haskell, F#) are associated with garbage collectors, runtime systems, and megabytes of memory. Why bring these ideas to an 8-bit CPU from 1976?

Because the *ideas* are independent of the *implementation costs*.

ML Concept	The Idea	Z80 Reality
ADT	Model data as tagged variants	A `u8` tag + payload. Same as C `enum` + `union`, but type-safe
Pattern matching	Dispatch on structure	

ML Concept	The Idea	Z80 Reality
		Compiles to `CP n / JR Z` chains — same as hand-written
Parametric polymorphism	Write once, use with any type	Monomorphized at compile time. Zero runtime cost
Effect tracking	Know which functions do I/O	Pure functions are provably side-effect-free. Compiler enforces it
Pipe operator	Data flows left-to-right	Compiles to direct `CALL` — no overhead
Property testing	Verify for ALL inputs	Runs 256 tests at *compile time*. Bugs caught before the binary exists

The key insight: **Frill has no runtime system**. No GC, no closures allocated on heap, no vtables, no boxed values. Every abstraction compiles to the same code you'd write in assembly. The abstractions exist only at compile time.

What Frill Has That C Doesn't

If you're writing Z80 programs in C (via SDCC), you already have structs, functions, and loops. What does Frill add?

1. **Pattern matching** — Exhaustive, compiler-checked dispatch. No forgotten `switch` cases.
2. **Pipe operator** — `x |> f |> g` reads left-to-right. No nested `g(f(x))`.
3. **Compile-time verification** — `assert fib 7 == 13` runs during compilation.
4. **Effect system** — `IO` annotation prevents accidental side effects in pure code.
5. **Property testing** — `prop |x| add x 0 == x` tests all 256 u8 values.
6. **Polymorphism** — `let id (x : 'a) = x` works for u8 AND u16 without duplication.
7. **Type-safe ADTs** — `type Color = Red | Green | Blue` with exhaustive matching.

Frill vs ML-Family Languages

How close is Frill to “real” ML? Honest comparison:

Feature	OCaml/ F#	Haskell	Frill	Status
Let bindings	`let x = 5 in x + 1`	Same	Same	**Identical syntax**
ADT	`type t = A | B of int`	`data T = A | B Int`	`type T = A | B of u8`	**Works**
Pattern matching	Full	Full	Match + guards	**Works** (no nested patterns yet)
Parametric polymorphism	`'a -> 'a`	`a -> a`	`'a -> 'a`	**Works** (monomorphized)
Type inference	Hindley-Milner	HM + extensions	Partial (params annotated)	**Partial**
Higher-order functions	First class	First class	Via pipes + compose	**Works** (no closures)
Pipe operator	F#: `|>`	N/A	`|>`	**Identical to F#**
Effect system	N/A	Monads	`IO` annotation	**Simpler than Haskell**
Linear types	N/A	Linear Haskell	QTT (`!`, `~`)	**Works**
Lazy evaluation	Option	Default	No	**By design** (strict)
Garbage collection	Yes	Yes	No	**By design** (no heap)
Modules	Yes	Yes	`import`	**Basic**
Type classes	N/A	Yes	`class`/ `instance`	**Partial**
Closures	Heap-allocated	Heap-allocated	No (lambda = named function)	**Limitation**

What's missing: nested pattern matching, recursive ADTs (lists), closures that capture mutable state, full Hindley-Milner inference, modules with signatures.

What's unique to Frill: compiles to Z80 assembly, property testing at compile time, QTT linearity annotations, zero runtime overhead.

Language Tour

Functions and Let Bindings

```
(* Functions: name, typed parameters, body *)
let add (a : u8) (b : u8) = a + b
let double (x : u8) = x + x

(* Let-in bindings — scoped values *)
let hypotenuse_sq (x : u8) (y : u8) =
  let xx = x * x in
  let yy = y * y in
  xx + yy

(* Compile-time verification *)
assert add 3 5 == 8
assert double 7 == 14
assert hypotenuse_sq 3 4 == 25
```

Recursion

```
let factorial (n : u8) =
  if n == 0 then 1
  else n * factorial (n - 1)

let fib (n : u8) =
  if n < 2 then n
  else fib (n - 1) + fib (n - 2)

assert factorial 5 == 120
assert fib 7 == 13
```

Pipe Operator and Composition

```
let inc (x : u8) = x + 1
let dbl (x : u8) = x * 2
let sq (x : u8) = x * x

(* Pipe: data flows left to right *)
let transform (x : u8) = x |> dbl |> inc
```

```
(* Composition: combine functions *)
let dbl_then_inc = dbl >> inc

assert transform 5 == 11      (* 5*2+1 = 11 *)
assert dbl_then_inc 5 == 11
```

Algebraic Data Types and Pattern Matching

```
type Color = Red | Green | Blue

let color_name (c : u8) =
  match c with
  | Red   -> 82   (* 'R' *)
  | Green -> 71   (* 'G' *)
  | Blue  -> 66   (* 'B' *)

type Day = Mon | Tue | Wed | Thu | Fri | Sat | Sun

let is_weekend (d : u8) =
  match d with
  | Sat -> 1
  | Sun -> 1
  | _   -> 0

assert color_name Red == 82
assert is_weekend Sat == 1
assert is_weekend Mon == 0
```

Parametric Polymorphism

```
(* Type variables: 'a is resolved at each call site *)
let id (x : 'a) : 'a = x
let poly_max (a : 'a) (b : 'a) : 'a = if a > b then a else b
let poly_clamp (x : 'a) (lo : 'a) (hi : 'a) : 'a =
  if x < lo then lo else if x > hi then hi else x

(* Each call generates a specialized u8 version *)
assert id 42 == 42
assert poly_max 10 20 == 20
assert poly_clamp 50 10 200 == 50
assert poly_clamp 5 10 200 == 10
```

Under the hood, `id 42` generates `id_u8(x: u8) -> u8 = x`. Same as Rust's monomorphization. Zero runtime dispatch, optimal register usage.

Effect System

```
(* Pure function — no side effects, safe for compile-time eval *)
let add (a : u8) (b : u8) = a + b

(* IO function — explicitly marked, can do I/O *)
let greet (x : u8) : IO u8 =
  do puts "Hello!\r\n"
  0

(* Compiler enforces: pure cannot call IO *)
(* let bad (x : u8) = do puts "oops"  -- ERROR! *)
```

The effect system is simpler than Haskell's monads. You mark functions `: IO` and the compiler checks that pure code stays pure. Extern functions (assembly, system calls) are implicitly IO.

Property-Based Testing

```
(* prop runs the predicate for ALL 256 u8 values *)
prop |x| add x 0 == x        (* identity *)
prop |x| poly_max x x == x   (* idempotent *)
prop |x| id x == x            (* polymorphic identity *)
```

Each `prop` generates 256 compile-time assertions. If any value fails, the compiler reports it before the binary is even created. This is exhaustive testing for u8 — not sampling, not heuristics, every single input.

While Loops and Mutation

```
let sum_to (n : u8) =
  let acc = 0 in
  let i = 0 in
  while i < n do
    do acc <- acc + i
    do i <- i + 1
  end
  acc

assert sum_to 10 == 45
```

Frill supports imperative loops where needed. The `do var <- expr` syntax makes mutation explicit — you always see where state changes.

String Operations

```
let char_at (s : u16) (n : u8) = peek (s + n)
let str_len (s : u16) : u8 =
```

```
  let p = 0 in
  while peek (s + p) > 0 do
    do p <- p + 1
  end
  p

assert char_at "Hello" 0 == 72   (* 'H' *)
assert char_at "Hello" 4 == 111  (* 'o' *)
assert str_len "Hello" == 5
```

Let-In Inside Expressions

```
(* let-in works everywhere — even inside if/else branches *)
let classify (x : u8) =
  if x > 100 then
    let half = x / 2 in half
  else
    let doubled = x * 2 in doubled

assert classify 200 == 100
assert classify 50 == 100
```

DSL Showcase

Expression Evaluator

A complete recursive-descent parser and evaluator in Frill, with correct operator precedence and parentheses:

```
let eval (src : u16) : u8 = unpack_val (eval_expr src 0)

assert eval "5" == 5
assert eval "3+4" == 7
assert eval "3+4*2" == 11     (* precedence: * before + *)
assert eval "2*(3+4)" == 14   (* parentheses *)
assert eval "(1+2)*3" == 9
```

17 compile-time assertions verify the evaluator handles every case correctly. The parser is ~60 lines of Frill, demonstrating that functional style makes recursive descent natural and readable.

Music DSL

Notes as algebraic data types, frequencies via pattern matching:

```
type Note = C | Cs | D | Ds | E | F | Fs | G | Gs | A | As | B |
Rest
```

```
let note_delay (n : u8) =
  match n with
  | C    -> 133   (* C4 = 262 Hz *)
  | E    -> 106   (* E4 = 330 Hz *)
  | G    -> 89    (* G4 = 392 Hz *)
  | A    -> 79    (* A4 = 440 Hz *)
  | Rest -> 0
  | ...

(* Melody encoding: note + duration packed into one byte *)
let encode (note : u8) (dur : u8) : u8 = note * 16 + dur

(* Verified: encode-decode round-trip *)
prop |x| decode_note (encode (x / 16) (x % 16)) == x / 16
```

33 asserts + 2 properties verify the entire music system at compile time.

Graphics Patterns

Pure functions that generate pixel data — Sierpinski triangles, XOR textures, checkerboards, smiley sprites:

```
(* Sierpinski: the classic bitwise fractal *)
let sierpinski (x : u8) (y : u8) =
  if (x & y) == 0 then 1 else 0

(* XOR texture: demoscene plasma effect *)
let xor_tex (x : u8) (y : u8) = (x ^ y) % 8

(* Compose into a multi-region display *)
let composite (x : u8) (y : u8) =
  if y < 32 then
    if x < 32 then sierpinski x y * 5
    else xor_tex x y
  else
    if checker x y 8 > 0 then 6 else 1
```

The pattern functions are pure and verified at compile time. The rendering loop runs on real Z80 hardware.

How It Compiles

Frill compiles through the full MinZ pipeline:

```
Frill (.frl) → HIR → MIR2 → VIR (Z3 solver) → Z80 Assembly →
Binary
```

Each step is a standard compiler transformation: - **Frill → HIR**: Desugar ML syntax into a typed intermediate representation - **HIR → MIR2**: Lower to SSA form with block arguments - **MIR2 → VIR**: Z3 SMT solver for joint instruction selection + register allocation (provably optimal) - **VIR → Z80**: Peephole optimization (16 rules), assembly emission - **Fallback**: PBQP register allocator for functions with inline assembly

A simple function like `let add (a : u8) (b : u8) = a + b` compiles to:

```
add:
    ADD A, B    ; a + b (result in A)
    RET         ; return
```

Two instructions. Zero overhead compared to hand-written assembly.

Compile-Time Verification

Frill's `assert` and `prop` statements run during compilation on the MIR2 virtual machine. This means:

- **Bugs are caught before the binary exists**
- **No test framework needed** — the compiler IS the test runner
- **Exhaustive for u8** — `prop` tests all 256 values, not a sample
- **Cross-verified** — same code runs on VM and Z80, both must agree

Current stats: **13 example files, 3351 compile-time checks, 0 failures**.

Turtle Graphics and L-Systems

Frill's pure functions and recursion make it natural for generative art. Here's a turtle graphics system with L-system fractal trees:

```
(* Turtle state — global for Z80 simplicity *)
let tx = 128
let ty = 160
let tangle = 192   (* pointing up in 256-step circle *)

(* Quarter-sine table: 64 entries, scale 112 *)
let fsin (a : u8) : i16 =
  let q = a / 64 in
  let idx = a & 63 in
  match q with
  | 0 -> qsin idx
  | 1 -> qsin (63 - idx)
  | 2 -> 0 - qsin idx
```

```
  | _ -> 0 - qsin (63 - idx)

let fcos (a : u8) : i16 = fsin (a + 64)

(* Move forward, drawing a line *)
let forward (dist : u8) =
  let nx = tx + fcos tangle * dist / 112 in
  let ny = ty + fsin tangle * dist / 112 in
  canvas_line tx ty nx ny color
  do tx <- nx
  do ty <- ny
```

Recursive Fractal Tree

```
(* Binary tree: trunk + two shorter branches at an angle *)
let branch (len : u8) (depth : u8) (angle : u8) =
  if depth == 0 then forward len
  else
    forward len
    let next = len * 2 / 3 in
    push ()
    left angle
    branch next (depth - 1) angle
    pop ()
    push ()
    right angle
    branch next (depth - 1) angle
    pop ()

(* Render: green tree, depth 7, 30-degree branching *)
let draw () =
  canvas_init 256 192 0
  canvas_clear 0
  do color <- 4
  branch 12 7 30
```

This generates fractal trees on a 256x192 ZX Spectrum canvas. The same code compiles to Z80 assembly and runs on real hardware.

L-System Forest

Five fractal trees rendered on MIR2 VM. Each tree uses different branching angles and depths. 256x192, ZX Spectrum palette.

Variations

Different parameters produce radically different trees:

Style	Parameters	Result
Symmetric	angle=28, depth=7	Classic binary tree
Ternary	3-way branch, depth=5	Bushy, dense canopy
Windblown	left=25, right=35	Asymmetric, natural look
Fern	alternating sides, depth=5	Barnsley fern pattern

Windblown Trees
Windblown Trees

Asymmetric trees — left branches longer than right, like wind-shaped growth.

Canvas and impl Blocks

Shapes can be drawn using `impl` blocks (OOP-style method dispatch):

```
(* In Nanz syntax — Frill uses the same HIR backend *)
struct Circle { cx : u16, cy : u16, r : u16, color : u16 }
struct Box { x : u16, y : u16, w : u16, h : u16, color : u16 }

impl Renderable for Circle {
    fun draw (self) -> u16 { canvas_circle self.cx self.cy self.r
self.color }
}

impl Renderable for Box {
    fun draw (self) -> u16 { canvas_fill_rect self.x self.y self.w
self.h self.color }
}
```

Impl Showcase
Impl Showcase

House scene rendered through impl dispatch: Circle (sun, tree crown), Box (sky, ground, house, door, window). All method calls resolve to direct CALL instructions at compile time — zero overhead.

Metrics

Metric	Value
Language features	41
Example programs	13+
Compile-time checks	3,351+

Metric	Value
Parser LOC (frill.go)	2,436
Stdlib modules	5 (math, tokenizer, I/O, functional, canvas)
Canvas output	6 PNG renders (L-system trees, impl shapes)
Compilation target	Z80 (ZX Spectrum, CP/M, Agon Light 2)

Getting Started

```
# Compile a Frill program
minzc hello.frl -b z80 -o hello.a80

# Compile for CP/M
minzc hello.frl -b z80 -t cpm -o hello.com

# All assertions are checked automatically during compilation
```

Create a file `hello.frl`:

```
let double (x : u8) = x + x
let inc (x : u8) = x + 1

assert double 5 == 10
assert inc 99 == 100
assert 5 |> double |> inc == 11

let main (x : u8) = 0
```

If every assert passes, you get a Z80 binary. If any fails, the compiler tells you exactly which assertion and what it got vs what was expected.

Appendix: Feature Reference

Feature	Syntax	Example
Function	`let name (p : type) = body`	`let inc (x : u8) = x + 1`
If-else	`if cond then a else b`	`if x > 0 then x else 0`
Let-in	`let x = e in body`	`let y = x * 2 in y + 1`
While	`while cond do ... end`	`while i < n do ... end`
Mutation	`do var <- expr`	`do i <- i + 1`

Feature	Syntax	Example
Pipe	`expr |> fn`	`5 |> double |> inc`
Compose	`f >> g`	`let h = inc >> double`
ADT	`type T = A | B | C`	`type Color = Red | Green | Blue`
Match	`match e with | P -> v`	`match c with | Red -> 2`
Polymorphism	`(x : 'a) : 'a`	`let id (x : 'a) : 'a = x`
Effect	`: IO type`	`let greet (x : u8) : IO u8 = ...`
Assert	`assert fn args == val`	`assert add 3 5 == 8`
Property	`prop |x| pred`	`prop |x| add x 0 == x`
Peek	`peek addr`	`peek (str + n)`
Import	`import "path"`	`import "../../stdlib/frill/math.frl"`
Extern	`extern name (p : t) : t`	`extern putchar (ch : u8) : u8`
Lambda	`|x| body`	`5 |> |x| x * 2`
String	`"text"`	`"Hello\r\n"`
Linearity	`(! x : t)` / `(~ x : t)`	`(! buf : u16)` (use once)

Chapter: Real-World Demos (v0.23.0)

Three demos that prove Frill is more than a toy — it's a practical language for Z80 development. Each one showcases a different strength of ML-style programming on constrained hardware.

Why Frill?

C can't do this. C has no ADTs, no exhaustive match, no pipe operators. A C state machine uses `switch` with no guarantee all states are handled — miss one and you get undefined behavior at runtime. On Z80, runtime UB means a hard crash with no debugger, no stack trace, no recovery. Frill catches it at compile time.

Assembly can't do this. Assembly has no types at all. A traffic light state is a number in a register — nothing stops you from passing 42 to a function that expects 0/1/2. Frill's type system prevents invalid states from existing.

Frill gives you ML safety with Z80 performance. ADTs compile to integer tags. Match compiles to conditional jumps. Pipe compiles to CALL chains. Zero overhead. The abstraction is free.

State Machine (175 bytes) — Why ADTs Matter

A traffic light controller with exhaustive state transitions:

```
type Light = Red | Yellow | Green

let next_light (s : u8) : u8 =
  match s with
  | Red    -> 2   (* → Green *)
  | Yellow -> 0   (* → Red *)
  | Green  -> 1   (* → Yellow *)
  end

let light_duration (s : u8) : u8 =
  match s with
  | Red    -> 30
  | Yellow -> 5
  | Green  -> 25
  end
```

Also includes a door lock (Locked/Unlocked/Open/Alarm) with key-based transitions. **32 assertions**, all verified at compile time.

The Z80 binary is 175 bytes. The compiler proves every state is handled — missing a match branch is a compile error, not a runtime crash.

File: `examples/frill/state_machine.frl`

Minigame Engine (226 bytes) — Why Pure Functions Matter

In C game engines, state is everywhere — global variables, mutable structs, pointer aliasing. Bugs hide in mutation. On Z80 with 64KB RAM and no debugger, mutable state bugs are fatal.

Frill's approach: **every game function is pure.** No mutation, no side effects. Entity classification, collision detection, damage calculation — all pure `u8 → u8` functions. The entire game state is recomputed each frame from immutable inputs.

This isn't slow — on Z80, pure functions compile to tight register-only code. And it's **GPU-parallelizable**: each entity's collision check runs independently on a GPU thread.

A complete game logic library using ADTs and pipe operators:

```
type Entity = Player | Enemy | Bullet | Coin | Wall

let is_solid (e : u8) : u8 =
  match e with
  | Player -> 0
  | Enemy  -> 1
  | Bullet -> 0
  | Coin   -> 0
  | Wall   -> 1
  end

let apply_score (base : u8) (combo : u8) : u8 =
  base * combo_mult combo

let tick_score (base : u8) : u8 =
  base |> double |> inc
```

Includes: entity system, Manhattan distance, 1D collision detection, combo scoring, health with damage/heal, pipe-based tick processing.

33 assertions. Every game logic function tested at compile time. 226 bytes on Z80.

File: `examples/frill/minigame.frl`

Parser Combinator (498 bytes)

Functional parsing on Z80 — character classification, tokenization, expression evaluation:

```
let parse_hex_byte (hi : u8) (lo : u8) : u8 =
  let h = parse_hex hi in
  let l = parse_hex lo in
  if h == 255 then 255
  else if l == 255 then 255
  else h * 16 + l

type Token = Number | Ident | Operator | LParen | RParen | Unknown
```

```
let classify_token (c : u8) : u8 =
  if is_digit c == 1 then 0
  else if is_alpha c == 1 then 1
  else if c == 43 then 2   (* '+' *)
  else if c == 40 then 3   (* '(' *)
  else if c == 41 then 4   (* ')' *)
  else 5

let eval_op (a : u8) (op : u8) (b : u8) : u8 =
  if op == 43 then a + b
  else if op == 45 then a - b
  else if op == 42 then a * b
  else if op == 47 then if b == 0 then 0 else a / b
  else 0
```

17 character classification functions (is_digit through to_upper), decimal and hex parsing, operator precedence, expression evaluation. **45 assertions.**

File: `examples/frill/parser_combinator.frl`

One Source, Five Targets

The same Frill code compiles to Z80 AND GPU. Here's `next_light` on both:

Frill source:

```
type Light = Red | Yellow | Green

let next_light (s : u8) : u8 =
  match s with
  | Red    -> 2   (* → Green *)
  | Yellow -> 0   (* → Red *)
  | Green  -> 1   (* → Yellow *)
  end
```

Z80 output (12 bytes, VIR Z3 optimal):

```
next_light:
    OR A                        ; test s == Red(0)?
    JR NZ, .cret_else2          ; no → check Yellow/Green
    LD A, 2                     ; Red → Green(2)
    RET
.cret_else2:
    CP 2                        ; test s == Green(2)?
    LD A, 0                     ; Yellow(1) → Red(0)
    RET Z                       ; if Green: return Yellow(1)...
```

```
    LD A, 1                         ; ...actually Green → Yellow(1)
    RET
```

CUDA output (same function, parallel on GPU):

```
__device__ uint8_t next_light(uint8_t s) {
    if (s == 0) return 2;  // Red → Green
    if (s == 2) return 1;  // Green → Yellow
    return 0;              // Yellow → Red
}
// All 3 states verified in parallel: 256/256 on NVIDIA GPU
```

Also compiles to: **OpenCL** (AMD/Intel), **Vulkan** (SPIR-V), **Metal** (Apple M2). All 4 GPU backends verified 256/256 on real hardware.

Frill vs Nanz — Same ASM, Different Syntax

The same function in both languages produces **identical Z80 assembly and identical CUDA kernel:**

	Frill	Nanz
Source	`let double (x : u8) : u8 = x + x`	`fun double(x: u8) -> u8 { return x + x }`
Z80	`ADD A, A / RET`	`ADD A, A / RET`
CUDA	`r2 = (r1 + r1) & 0xFF;`	`r2 = (r1 + r1) & 0xFF;`

Both frontends lower to the same MIR2. MIR2 is the universal bridge — any of the 8 frontends can target any of the 5 backends. Choose your syntax, get the same optimal code.

Compile to all 5 backends from command line:

```
        mz demo.frl -b z80     -o demo.a80      # Z80 assembly (2 bytes!)
        mz demo.frl -b cuda    -o demo.cu       # NVIDIA CUDA kernel
        mz demo.frl -b opencl  -o demo.cl       # OpenCL (AMD/Intel)
        mz demo.frl -b vulkan  -o demo.comp     # Vulkan GLSL compute
shader
        mz demo.frl -b metal   -o demo.metal    # Apple Metal shader
```

Compilation Pipeline

All three demos compile through the VIR backend (Z3 SMT solver):

Demo	Z3 functions	PBQP fallback	Binary
state_machine	9/9	0	175 bytes

Demo	Z3 functions	PBQP fallback	Binary
minigame	14/18	4	226 bytes
parser_combinator	13/18	5	498 bytes

Functions that exceed Z3's timeout (30s) or require parameter register swapping fall back to PBQP heuristic allocation. The result is always correct — the question is only whether it's provably *optimal*.

Assert Summary

Demo	Asserts	Topics
state_machine	32	ADT transitions, exhaustive match
minigame	33	Entity ADT, collision, scoring, health, pipe
parser_combinator	45	Char classify, digit parse, tokenize, eval
Total new	**110**	
Frill total	**427**	Across 16 examples